



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

Nutri Planner Application

Name: Timbuc Iulia
Group: 30237

Table of Contents

<i>Deliverable 1</i>	3
Project Specification	3
Functional Requirements	3
Use Case Model	3
Use Cases Identification:	3
UML Use Case Diagrams	4
Supplementary Specification	5
Non-functional Requirements.....	5
Design Constraints	6
Glossary	6
<i>Deliverable 2</i>	6
Domain Model	6
Architectural Design	7
Conceptual Architecture	7
Package Design.....	8
Component and Deployment Diagram	9
<i>Deliverable 3</i>	10
Design Model	10
Dynamic Behavior	10
Class Diagram	10
Data Model	10
<i>System Testing</i>	10
<i>Future Improvements</i>	10
<i>Conclusion</i>	10
<i>Bibliography</i>	10

Deliverable 1

Project Specification

NutriPlanner este o aplicație web Spring MVC pentru planificarea nutrițională și urmărirea sănătății. Permite utilizatorilor să-și gestioneze jurnalele alimentare zilnice, consumul de apă, greutatea și obiectivele nutriționale personalizate.

Functional Requirements

User - Crearea, editarea și stergerea profilurilor de utilizator. Fiecare utilizator are date personale (nume, email, vârstă, gen, înălțime, greutate), obiectiv (LOSE_WEIGHT, GAIN_WEIGHT, GAIN_MUSCLE, MAINTAIN) și nivel de activitate (SEDENTARY, LIGHTLY_ACTIVE, MODERATELY_ACTIVE, VERY_ACTIVE, EXTRA_ACTIVE). La creare/actualizare, sistemul calculează automat obiectivele zilnice de calorii, proteine, carbohidrați și grăsimi folosind formula Mifflin-St Jeor + TDEE.

Food - Gestionarea unui catalog de alimente cu macronutrienți (calorii, proteine, carbohidrați, grăsimi), unitate de măsură și dimensiune porție.

Daily food log - Logarea alimentelor consumate pe tipuri de masă (BREAKFAST, LUNCH, DINNER, SNACK). Valorile nutriționale se calculează automat proporțional cu cantitatea introdusă față de porția standard. Utilizatorul vede totalurile zilnice și pe masă, cu posibilitate de editare și ștergere, doar pentru ziua curentă.

Water log — Inregistrarea consumului de apă în ml. Afisează totalul zilnic, istoricul pe date și toate intrările din ziua curentă.

Weight log — Logarea greutății zilnice. Calculează automat BMI, diferența față de greutatea inițială și față de greutatea țintă. Afisează istoricul complet sortat cronologic.

Use Case Model

Use Cases Identification:

Use-Case: Adăugare alimente în fiecare zi

Level: User Goal

Primary Actor: User

Main success scenario: După logare, user-ului îi apare pagina unde poate adăuga alimente pentru ziua curentă, grupate pe tipul de masă (breakfast, lunch, dinner and snack). User-ul alege tipul de masă, iar apoi selectează un aliment din catalog, introduce apoi cantitatea consumată. Sistemul calculează valorile nutriționale proporțional cu porția standard. Sistemul salvează introducerea și e redirecționează la pagina zilnică actualizată. User-ul vede totalul zilnic actualizat pentru calorii, proteine, carbohidrați și grăsimi, dar și pe tipul de masă.

Extensions: Dacă catalogul de alimente este gol, atunci user-ul trebuie prima dată să creeze alimente. Cantitatea unui aliment e 0 sau negativă, sistemul salvează date invalide negative. Dacă user-ul vrea să ștergă ceva, este permisă doar ștergerea pentru ziua curentă.

Use-Case: Inregistrarea unui nou user si calcularea obiectivului zilnic

Level: User Goal

Primary Actor: User

Main success scenario: User-ul completeaza formularul de inregistrare cu datele lui personale (nume, email, parola, varsta, gen, inaltime, greutate, obiectiv, nivel de activitate). Sistemul verifica daca mailul nu a mai fost folosit. Sistemul calculeaza BMR folosind formula Mifflin-St Jeor pe baza genului, greutatii, inaltimii si varstei. Sistemul aplica multiplicatorul in functie de nivelul de activitate. Apoi ajusteaza caloriile in functie de obiectiv. Se calculeaza targeturile zilnice de calorii protein, grasimi si carbohidrati. User-ul este directionat catre pagina principala.

Extensions: Daca email-ul exista deja, sistemul revine inapoi la pagina aia si trimite un mesaj de eroare cum ca email-ul e deja existent. Daca un utilizator selecteaza ca vrea sa slabeasca, dar pune ca obiectiv de kg un numar mai mare decat kg lui initiale, sistemul va calcula ceva gresit.

Use-Case: Urmarirea progresului

Level: User Goal

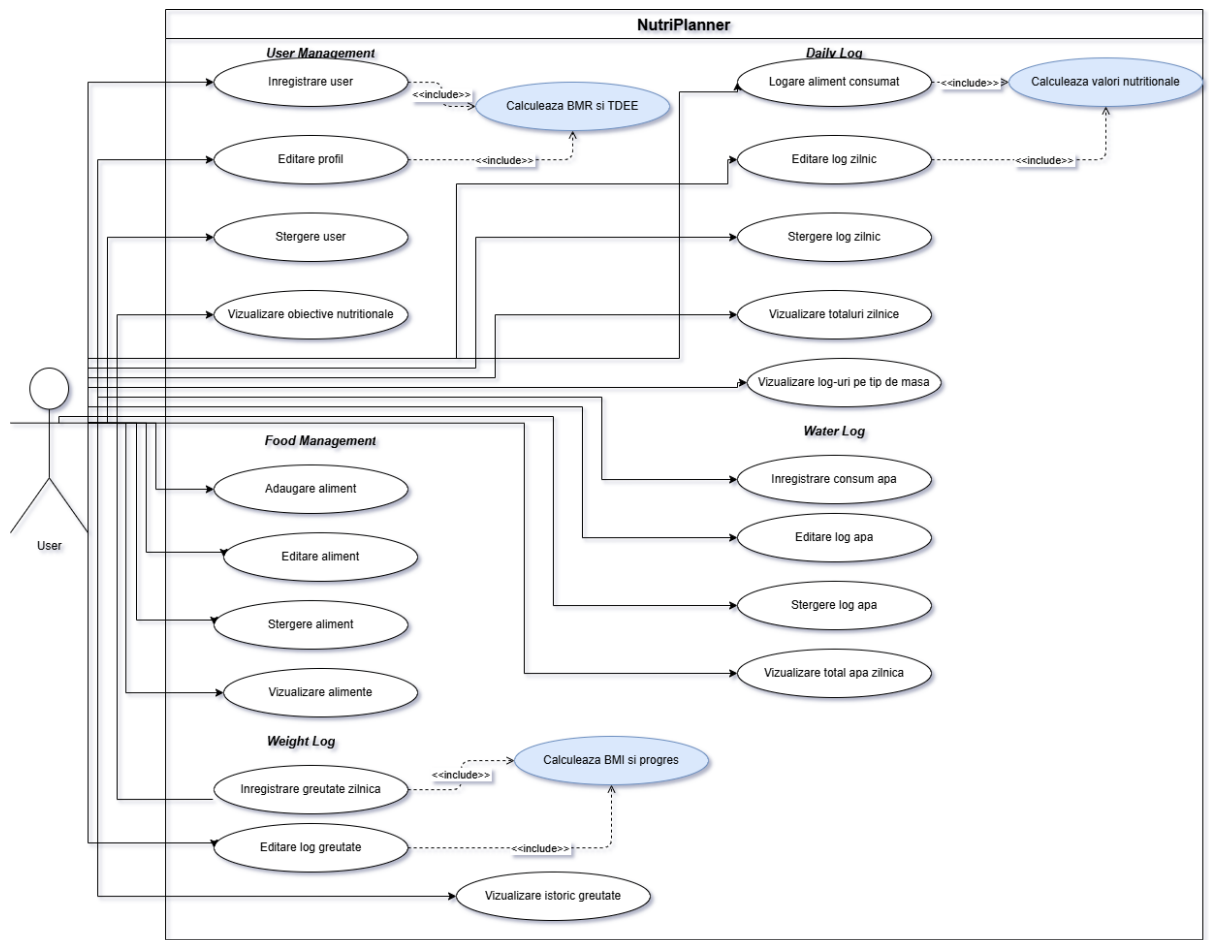
Primary Actor: User

Main success scenario: User-ul inregistreaza greutatea curenta pentru ziua curenta. Sistemul verifica ca nu exista deja o inregistrare de greutate pentru ziua de azi. Apoi, calculeaza BMI folosind inaltimea din profilul userului, iar acesta va fi afisat undeva. Sistemul preia cea mai curenta introducere de greutate pentru a calcula diferenta fata de greutatea de start apoi, calculeaza diferenta fata de greutatea tinta, pentru a stii cat mai avem pana la progresul final. Se salveaza inregistrarea iar, userul poate vizualiza istoricul complet sortat crescator dupa data.

Extensions: Un log de greutate pentru azi exista deja sistemul arunca o exceptie. Nu exista introduceri anterioare sau user-ul nu a setat o greutate tinta, iar atunci diferenta fata de targhet se calculeaza cu o valoare implicita, iar rezultatele vor fi incorecte.

UML Use Case Diagrams

[Draw the diagrams for the entire application.]



Supplementary Specification

Non-functional Requirements

[Choose 4 NF for your system, describe them and explain why these NF are suitable for your implementation.]

1. Utilizabilitate

Aplicatia trebuie sa fie usor de folosit de catre orice utilizator, fara cunostinte tehnice. Interfata bazata pe Spring MVC cu formulare simple si redirectionari clare asigura o navigare intuitiva.

2. Fiabilitate

Sistemul trebuie sa gestioneze corect erorile si sa nu piarda date. In implementare se folosesc exceptii explicite pentru cazuri invalide, cum ar fi stergerea unei inregistrari dintr-o zi trecuta sau inregistrarea a doua greutati in aceeasi zi. Este potrivita deoarece un calcul gresit al caloriilor sau o inregistrare duplicata poate afecta deciziile utilizatorului.

3. Securitate

Datele utilizatorului sunt personale, aplicatia stocheaza informatii precum greutatea, dieta, obiectivele corporale si datele medicale indirecte (BMI, calorii). Acestea sunt date private pe care un utilizator nu ar vrea sa le vada altcineva si trebuie securizate prin autentificare cu mail si parola.

4. Mentenabilitate

Codul trebuie sa fie usor de extins si modificat. Arhitectura pe layere (Controller, Service, Repository) si separarea clara a responsabilitatilor faciliteaza adaugarea de functionalitati noi fara a afecta codul existent. Este potrivita deoarece aplicatia este in dezvoltare activa si sunt necesare imbunatatiri.

Design Constraints

Aplicatia NutriPlanner este dezvoltata in Java, folosind framework-ul Spring Boot cu arhitectura MVC. Aceasta arhitectura impune separarea codului in trei layere: Controller, Service si Repository, care trebuie respectata pe toata durata dezvoltarii.

Datele sunt stocate in memorie , ceea ce inseamna ca toate informatiile se pierd la repornirea aplicatiei. Aceasta este o constrangere actuala a sistemului, care se poate modifica pe parcurs.

Interfata utilizatorului este construita cu Thymeleaf si HTML, impunand ca toate view-urile sa fie fisiere .html gestionate de template engine-ul Thymeleaf.

Clasele model folosesc libraria Lombok (@Getter, @Setter), care trebuie sa fie prezenta obligatoriu in proiect pentru ca acesta sa compileze. Managementul dependintelor se face prin Maven (pom.xml).

Glossary

[Present the noteworthy terms and their definition, format and validation rules if appropriate.]

User - utilizatorul aplicatiei.

BMR (Basal Metabolic Rate) - reprezinta numarul de calorii pe care corpul le consuma in repaus complet. Calculat prin formula Mifflin-St Jeor.

TDEE (Total Daily Energy Expenditure) - totalul de calorii consumate intr-o zi, calculat prin inmultirea BMR cu un multiplicator specific nivelului de activitate fizica.

DailyLog — o inregistrare zilnica a unui aliment consumat de un utilizator. Contine

BMI (Body Mass Index) - indicele de masa corporala, calculat ca greutate (kg) / inaltime (m)^2. Folosit pentru a evalua daca greutatea unui utilizator este in limite normale.

Deliverable 2

Domain Model

[Define the domain model and create the conceptual class diagrams]

User - reprezinta utilizatorul aplicatiei. Attribute: id, name, email, password, age, gender, weight, initialWeight, height, targetWeight, targetDate, goal, activityLevel, dailyCalorieGoal, dailyProteinGoal, dailyCarbsGoal, dailyFatGoal. Pe baza datelor introduse (greutate, inaltime, varsta, obiectiv, nivel de activitate) se calculeaza automat obiectivele zilnice de calorii si macronutrienti folosind formula Mifflin-St Jeor.

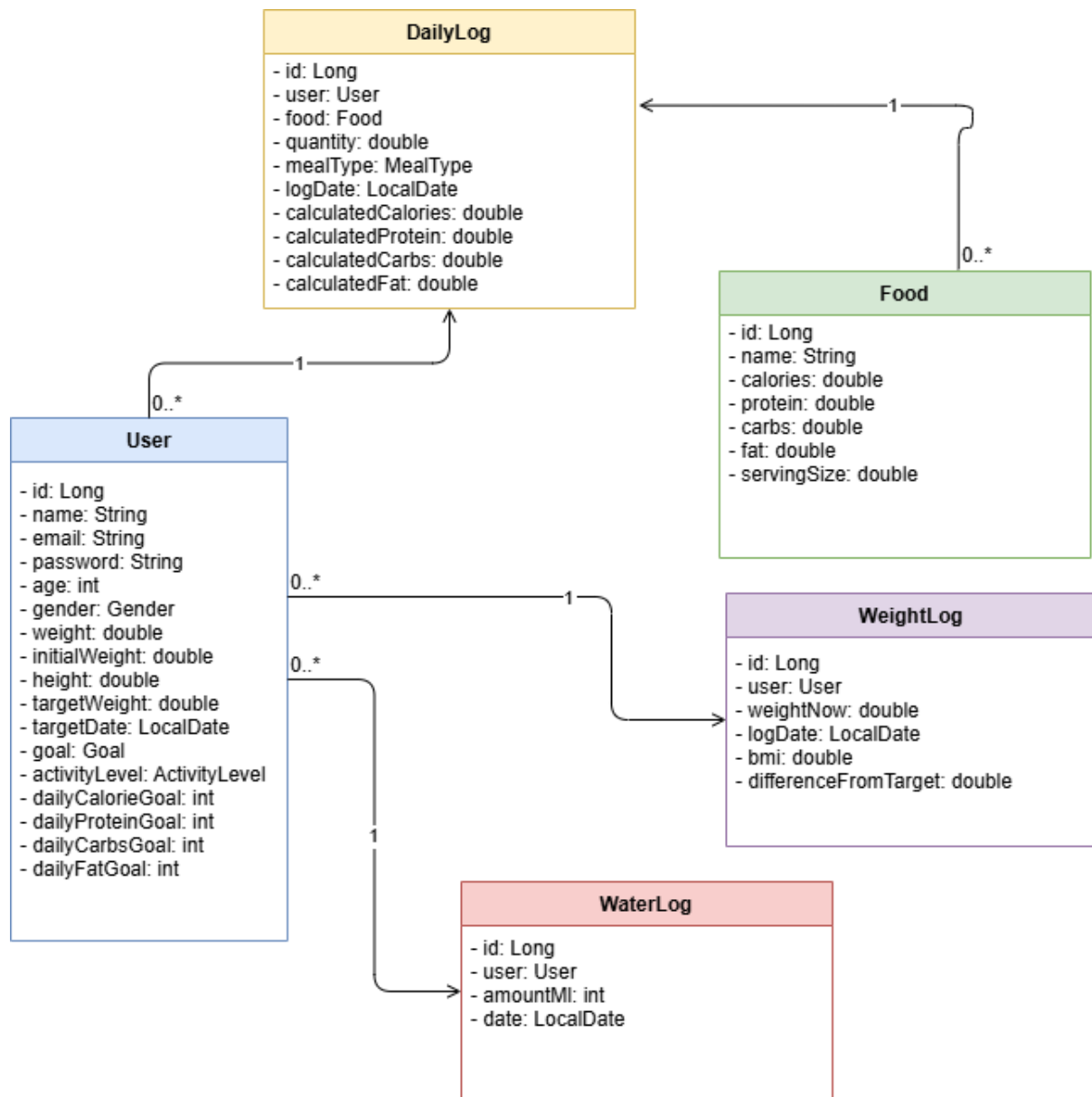
Food - reprezinta un aliment. Attribute: id, name, calories, protein, carbs, fat, servingSize. Valorile nutritionale sunt raportate la portia standard (servingSize).

DailyLog - reprezinta un aliment consumat de un user la o anumita masa si la o anumita data. Attribute: id, user, food, quantity, mealType, logDate, calculatedCalories, calculatedProtein, calculatedCarbs, calculatedFat. Valorile nutritionale sunt calculate proportional cu cantitatea consumata fata de portia standard.

WaterLog - reprezinta o inregistrare a consumului de apa al unui user. Attribute: id, user, amountML, date. Se insumeaza toate inregistrarile din ziua respectiva pentru a obtine totalul zilnic.

WeightLog - reprezinta o inregistrare a greutatii unui user la o anumita data. Attribute: id, user, weightNow, logDate, bmi, differenceFromTarget. BMI-ul si diferenta fata de greutatea tinta sunt

calculate automat la salvare.



Architectural Design

Conceptual Architecture

[Define the system's conceptual architecture; use an architectural style and pattern - highlight its use and motivate your choice]

Presentation Layer - reprezinta interfata utilizatorului implementata in React. Contine paginile Login, Register, Home, MealDetail, AddFood si WeightProgress. Comunica cu backend-ul exclusiv prin requesturi HTTP REST folosind Axios.

Controller Layer - contine clasele adnotate cu `@RestController` care primesc requesturile HTTP de la frontend si returneaza raspunsuri JSON. Fiecare controller gestioneaza un domeniu specific: *AuthController*, *UserController*, *FoodController*, *DailyLogController*, *WaterLogController*, *WeightLogController*.

Service Layer - contine logica de business a aplicatiei. Aici se calculeaza obiectivele nutritionale zilnice folosind formula Mifflin-St Jeor, se calculeaza BMI-ul, se valideaza datele si se coordoneaza operatiile intre repository-uri.

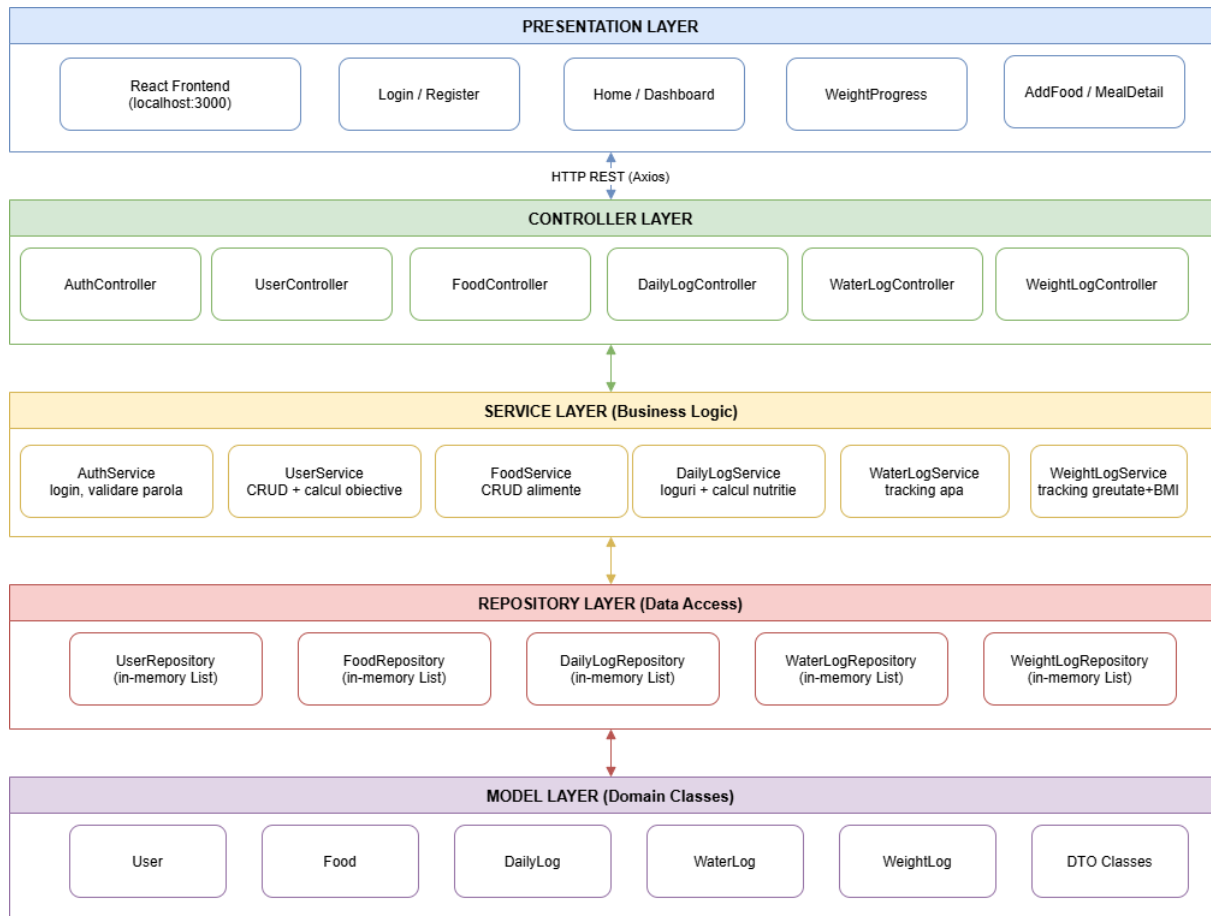
Repository Layer - gestioneaza accesul la date. Datele sunt stocate in memorie folosind liste Java (in-

memory storage).

Model Layer - contine clasele de domeniu (User, Food, DailyLog, WaterLog, WeightLog) si clasele DTO folosite pentru transferul datelor intre straturi.

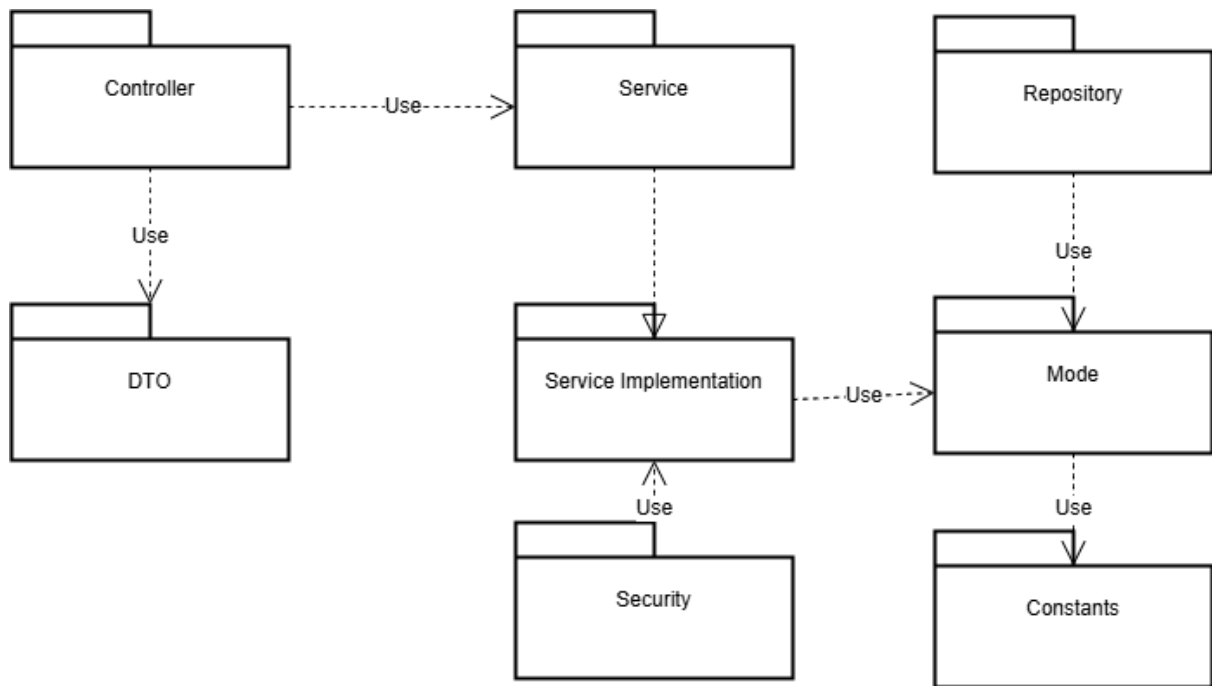
Architecture a fost aleasa deoarece:

- Separa clar responsabilitatile fiecarui strat, facand codul mai usor de inteles si de intretinut
- Permite modificarea unui strat fara a afecta celelalte straturi
- Faciliteaza testarea unitara a fiecarui strat in mod independent
- Este un standard recunoscut in dezvoltarea aplicatiilor enterprise cu Spring Boot



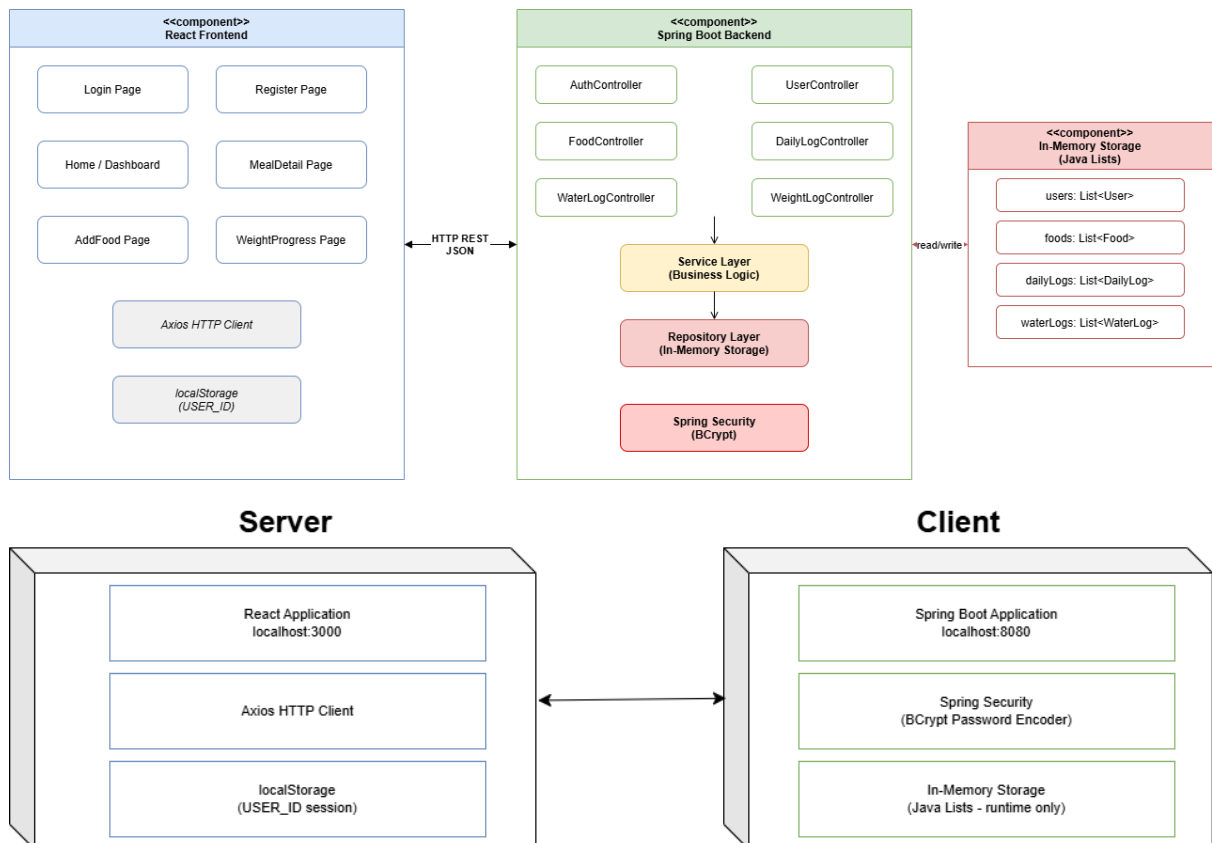
Package Design

[Create a package diagram]



Component and Deployment Diagram

[Create the component and deployment diagrams.]



Deliverable 3

Design Model

Dynamic Behavior

[Create the interaction diagrams (2 sequence) for 2 relevant scenarios]

Class Diagram

[Create the UML class diagram; apply GoF patterns and motivate your choice]

Data Model

[Create the data model for the system.]

System Testing

[Describe the testing methods and some test cases.]

Future Improvements

[Present some features that apply to the application scope.]

Conclusion

Bibliography